Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T23 **Semester:** V

Student Name: Anirudh Vineet **Roll No:** 2401124

EXPERIMENT NO. 07

Aim: - To implement monitoring using Prometheus / Grafana / Nagios.

Theory:

In modern DevOps environments, continuously monitoring the health, performance, and metrics of infrastructure and applications is critical. This experiment focuses on setting up a widely used open-source monitoring stack: Prometheus and Grafana.

Prometheus is a powerful time-series database and monitoring system. It works on a "pull" model, meaning it actively scrapes (fetches) metrics from configured endpoints via HTTP at regular intervals. It stores this data and allows users to query it using PromQL (Prometheus Query Language).

Grafana is an open-source analytics and interactive visualization web application. It connects to various data sources (including Prometheus) and translates the raw metrics into readable, highly customizable charts, graphs, and dashboards.

Together, they form a complete monitoring solution: Prometheus collects and stores the data, while Grafana visualizes it for the end user.

IMPLEMENTATION

Step 1: Creating the Configuration Files

A working directory was created, and the required configuration files were generated to deploy the stack using Docker Compose.

First, a prometheus.yml file was created to configure the scraping targets. In this experiment, Prometheus was configured to monitor its own internal metrics on port 9090.

```
global:
  scrape_interval: 15s

scrape_configs:
```

```
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
```

Next, a docker-compose.yml file was created to define and orchestrate both the Prometheus container and the Grafana container, exposing ports 9090 and 3000 respectively.

Step 2: Starting the Monitoring Stack

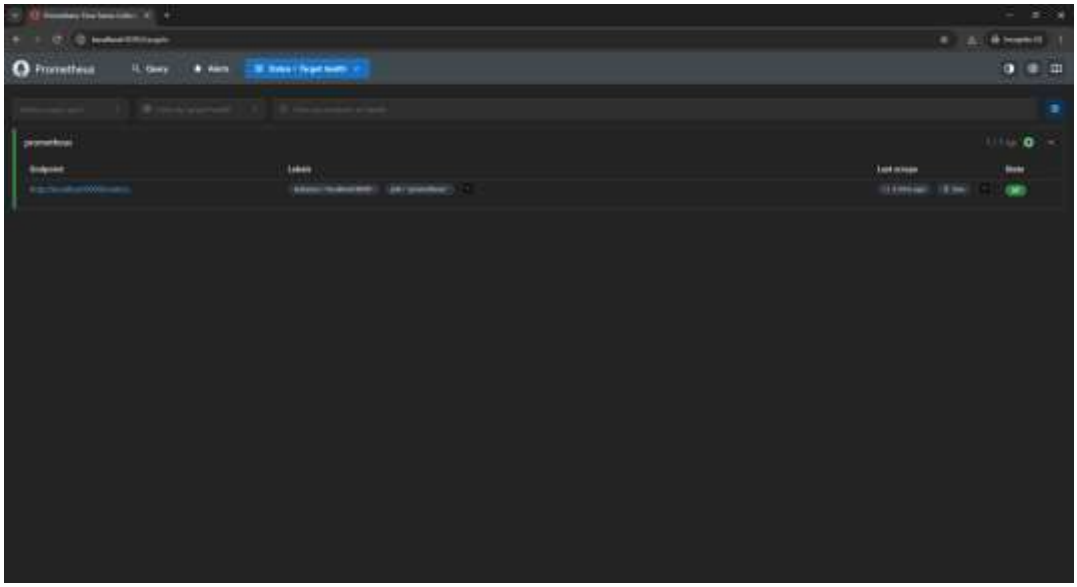
The containers were launched in detached mode using Docker Compose:

```
docker compose up -d
```

Docker successfully pulled the required images from Docker Hub and started the interconnected Prometheus and Grafana services.

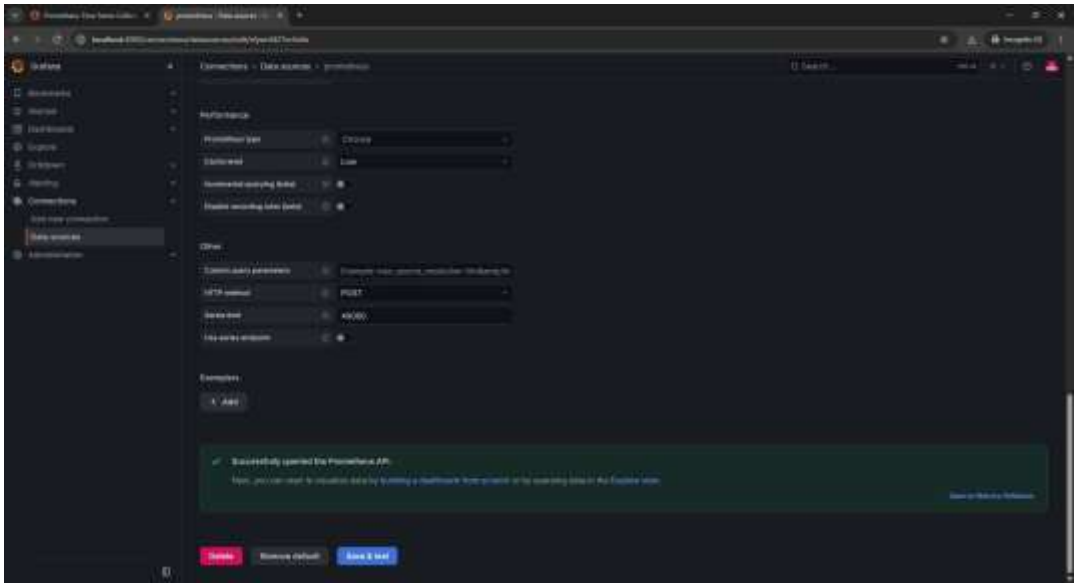
Step 3: Verifying Prometheus Target Status

The Prometheus web interface was accessed via http://localhost:9090. Navigating to the **Status > Targets** page confirmed that Prometheus was successfully scraping its internal metrics endpoint, showing a green **UP** state.



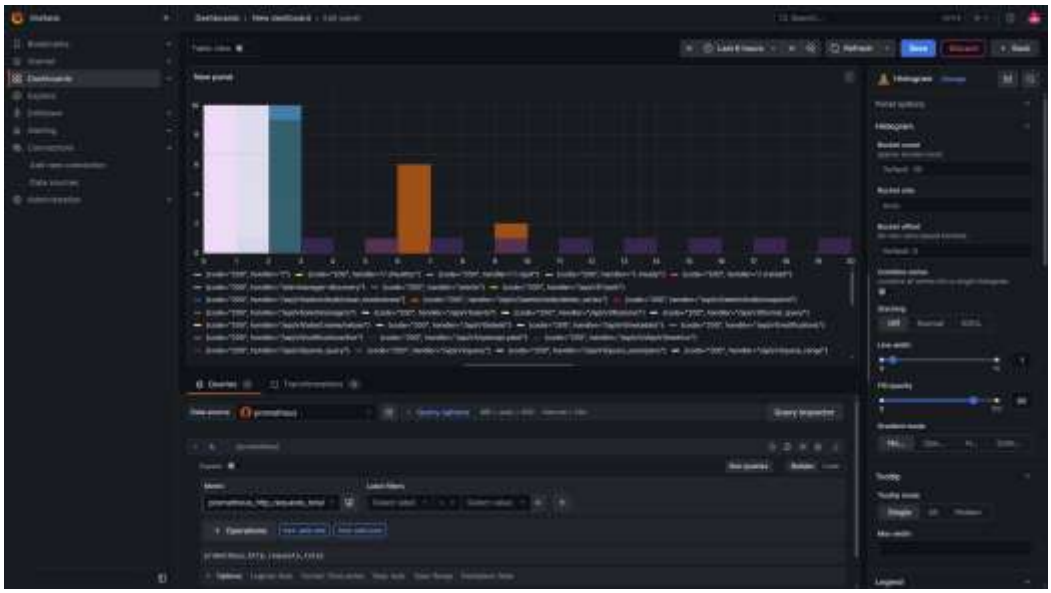
Step 4: Configuring Grafana Data Source

The Grafana dashboard was accessed via http://localhost:3000 and logged into using the default admin credentials. To allow Grafana to fetch data, a new **Data Source** was added. Prometheus was selected, and the internal Docker network URL (http://prometheus:9090) was provided. The connection was successfully verified with a "Successfully queried the Prometheus API" message.



Step 5: Creating the Visualization Dashboard

With the data source connected, a new dashboard was created to visualize the live traffic of the server. A visualization panel was added, and the PromQL query prometheus_http_requests_total was executed. Grafana successfully generated a histogram visualizing the total HTTP requests handled by the Prometheus server over time.



Conclusion: Thus, we have successfully studied the concepts of system monitoring and utilized Docker Compose to deploy, configure, and integrate a complete Prometheus and Grafana monitoring stack to scrape and visualize server metrics.